

https://drive.google.com/file/d/1sCH_co_DoMinDPF-jJnllgCRnSQwEjF5/view

[Lec-15: Difference between LALR & CLR parsing table | Imp. Example | Compiler Design](#)

[Lec-14: LALR Parsing Table | LALR vs CLR | Compiler Design](#)

[Lec-13: CLR Parsing Table | LR\(1\) Canonical Items](#)

◆ Why Do We Need More Powerful LR Parsers?

✗ Problem with SLR(1)

- Uses **FOLLOW(A)** for reduction
- Sometimes too **general (too broad)**
- Causes **wrong reductions** → **conflicts**

✓ **Goal:** Make reduction decisions **more precise**

◆ Two More Powerful Methods

★ 1. Canonical LR(1) / CLR (1) / LR(1)

- Uses **full lookahead information**
 - **Most powerful** LR parser
 - Uses **LR(1) items** instead of LR(0)
-

★ 2. LALR(1)

- Combines:
 - **LR(0)** structure
 - **some lookahead info**

📌 Key Feature

- **Fewer states** than LR(1)
- Almost as powerful as LR(1)
- Same size as **SLR tables**

LALR is used in real compilers (like YACC)

◆ Core Problem Explained

In SLR: **Reduce** $A \rightarrow \alpha$ if $a \in \text{FOLLOW}(A)$

✗ Problem

Even if: $a \in \text{FOLLOW}(A)$

👉 It might still be **invalid in that specific situation**

💡 Why?

Because:

- **FOLLOW(A)** is **global**
 - But parsing needs **local context**
-

◆ Solution Idea

Add more information to states → make decisions context-aware

◆ Canonical LR(1) / CLR(1) / LR(1)

› An LR(1) item is an LR(0) item paired with a lookahead terminal

Instead of: $A \rightarrow \alpha \cdot \beta$

We use: $[A \rightarrow \alpha \cdot \beta, \alpha]$

◆ What is “ α ”?

👉 Lookahead symbol

- Tells: “When should this reduction happen?”
-

◆ Meaning of LR(1) Item

📌 Case 1:

$[A \rightarrow \alpha \cdot \beta, \alpha]$

👉 Dot NOT at end

👉 Lookahead is ignored for now

Case 2

$$[A \rightarrow \alpha \cdot, a]$$

👉 Dot at end

✓ Reduce **ONLY** if next input = a

LR(1) reduces only when **lookahead matches**

◆ Comparison: SLR vs LR(1)

Feature	SLR(1)	LR(1)
Reduce condition	FOLLOW(A)	Specific lookahead
Precision	Low	High
Conflicts	More	Very few
States	Fewer	Many more

◆ Why LR(1) is Better

- Avoids invalid reductions
 - Uses exact context
 - Eliminates many conflicts
-

♦ Important Statement Explained

"The set of such α 's is a **subset of FOLLOW(A)**"

Meaning:

- LR(1) uses:
 - 👉 **smaller, more accurate set**
 - Instead of:
 - 👉 full FOLLOW(A)
-

♦ Formal Definition (Simplified)

Item:

$[A \rightarrow \alpha \cdot \beta, \alpha]$

Means:

👉 We are parsing:

- Already seen: α
 - Expect: β
 - And **after finishing**:
 - 👉 **Next symbol** must be α
-

♦ When is Reduction Valid?

For: $[A \rightarrow \alpha \cdot, \alpha]$

👉 Reduce ONLY if: **Next input = α**

◆ LALR(1) (Quick Idea)

✓ What it does:

- Starts from LR(1)
 - Merges similar states
-

✓ Result:

- Smaller table (like SLR)
- More powerful (close to LR)

16. Summary: Comparison of LR Parsers

The table below summarizes the key differences between the four main LR parser types:

Parser	Lookahead	Item Type	Table Size	Power
LR(0)	None	LR(0)	Small	Weakest
SLR(1)	FOLLOW set	LR(0)	Small	Moderate
CLR(1)	Specific lookahead	LR(1)	Large	Strongest
LALR(1)	Merged lookahead	LR(1) merged	Medium	Practical